UT4: Objetos predefinidos

Desarrollo de Aplicaciones Web Entorno-Cliente

Una pequeña introducción

Profundizaremos en los conceptos clave introducidos en las unidades temáticas anteriores:

BOM

Browser Object Model

Permite la interacción directa con el navegador fuera del contenido de la página.

Componentes clave:

- **window:** Objeto global principal
- **location:** Control de URL y navegación
- **history:** Historial de la pestaña
- **navigator:** Información del navegador
- **screen:** Propiedades de la pantalla

DOM

Document Object Model

Estructura jerárquica en árbol que representa los elementos de la página web.

Características principales:

- **document:** Punto de entrada al HTML
- Representación de etiquetas como nodos
- Manipulación dinámica con **JavaScript**
- Integración nativa con **HTML5**
- Acceso a eventos y estilos dinámicos

Contenido

1.- Objeto Window

01



Document

Representa la página web cargada en el navegador.

Punto de entrada principal para la manipulación del DOM.

02



Navigator

Información sobre la aplicación del navegador.

Permite consultar el idioma, geolocalización y estado.

03



Screen

Proporciona propiedades del monitor del usuario.

Dimensiones de pantalla, resolución y profundidad de color.

04



History

Contiene el historial de navegación de la pestaña actual.

Permite avanzar o retroceder entre páginas visitadas.

05



Location

Ofrece información sobre la URL actual.

Permite redireccionar a nuevas direcciones web.

1. El objeto Window

DEFINICIÓN

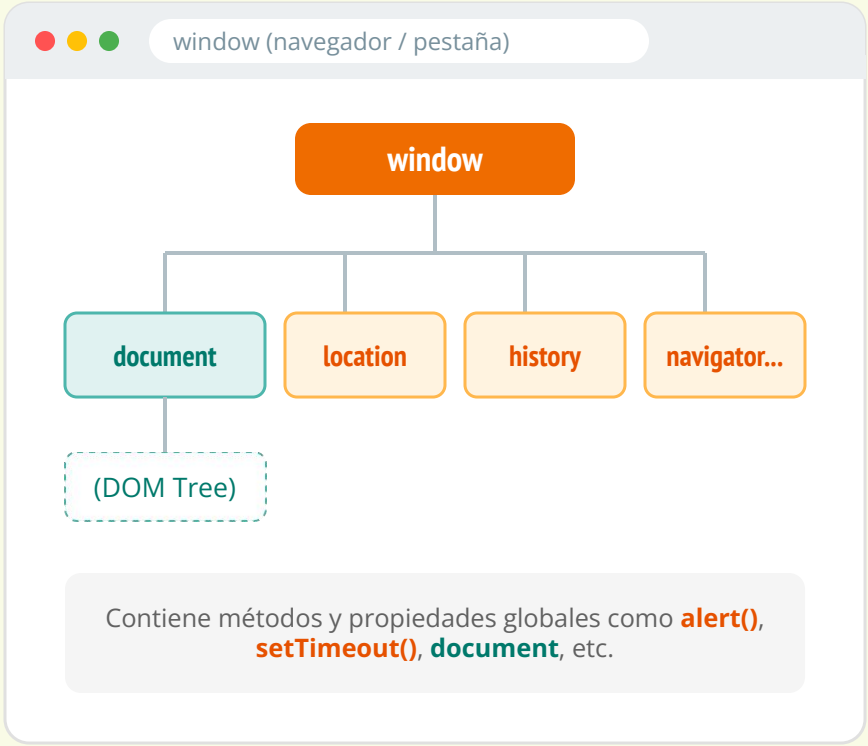
Objeto Global del Navegador

Representa la ventana o pestaña activa del navegador donde se está ejecutando tu página web.

ALCANCE

Cúspide de la Jerarquía

Casi todo lo que hacemos en el navegador cuelga directamente de él (DOM, BOM, variables y funciones globales).



1. Objeto Window

A la hora de llamar a los métodos o propiedades que posee el objeto window, podríamos hacerlo de forma explícita, por ejemplo:

DOCUMENT

```
document === window.document
// true
```

LOCATION

```
location === window.location
// true
```

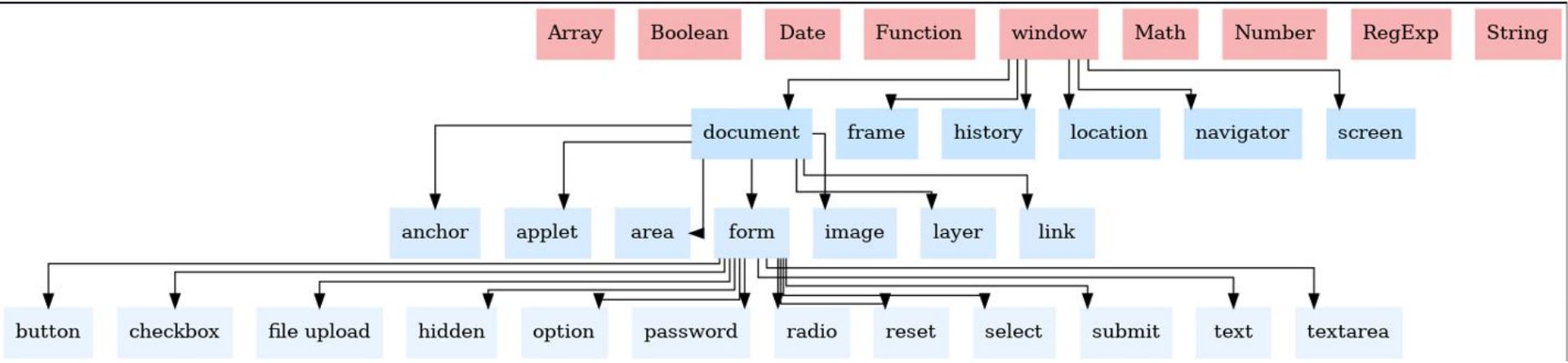
HISTORY

```
history === window.history
// true
```

ECONOMÍA DE CÓDIGO

Sin embargo, por economía de código, las llamadas las hacemos sin **window** delante, ya que funciona exactamente igual.

Jerarquía de Objetos



Jerarquía de Objetos

Dentro del entorno del navegador existen dos modelos fundamentales:

BOM

Browser Object Model

Es el modelo de objetos del navegador. Permite acceder y modificar las propiedades de las ventanas y del propio entorno del navegador.

DOM

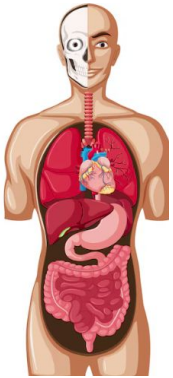
Document Object Model

Es el modelo de objetos del documento HTML. Representa y permite manipular la estructura jerárquica de contenidos de la página web.

ANALOGÍA DE ANATOMÍA

Si **BOM** y **DOM** se trataran de un cuerpo humano:

- **BOM** sería el **cuerpo y la estructura general**.
- **DOM** serían todos los **óganos internos** que lo mantienen funcionando.



Jerarquía de Objetos

BOM (Browser Object Model)

ORIGEN

Origen e Historia

Internet Explorer y Netscape Navigator introdujeron el concepto de Browser Object Model (BOM).

PROPIEDADES

Gestión de Ventanas

Permite acceder y modificar de forma dinámica las propiedades de las ventanas del propio navegador.

ESTRUCTURA

Objeto Window

El modelo BOM está principalmente constituido por el objeto raíz **window** y todas sus propiedades.

NO ESTÁNDAR

Interoperabilidad

A diferencia de DOM, ninguna entidad estandariza el BOM, existiendo diferencias entre navegadores.

DOCUMENTACIÓN Y REFERENCIAS: JavaScript References | Document Object Model (DOM)

BOM - Modelo de Objeto del Navegador



Introducción

Internet Explorer y Netscape Navigator introdujeron el concepto de Modelo de Objetos del Navegador.



Propiedades de la ventana

El BOM permite acceder y modificar las propiedades de las ventanas del propio navegador.



Objeto window

El modelo BOM lo constituyen el objeto window y todas sus propiedades.



No estándar

El mayor inconveniente de BOM es que ninguna entidad se encarga de estandarizarlo.

OBJETOS DEL NAVEGADOR (BOM)

Interacción de los Objetos con el navegador

CONCEPTO PRINCIPAL

Además de los objetos presentados anteriormente, existe otro tipo de objetos que permiten manipular diferentes características del navegador en sí mismo.

Ámbitos de interacción:

- Control de ventanas, marcos y pantallas
- Gestión de la navegación e historial de URLs
- Consulta de propiedades del entorno de ejecución



Jerarquía de Objetos

DOM (Document Object Model)

ESTÁNDAR W3C

Acceso al Documento

Estándar W3C para acceder al objeto Document y a todas sus propiedades (objetos que representan etiquetas HTML).

ESTRUCTURA

Árbol de Nodos

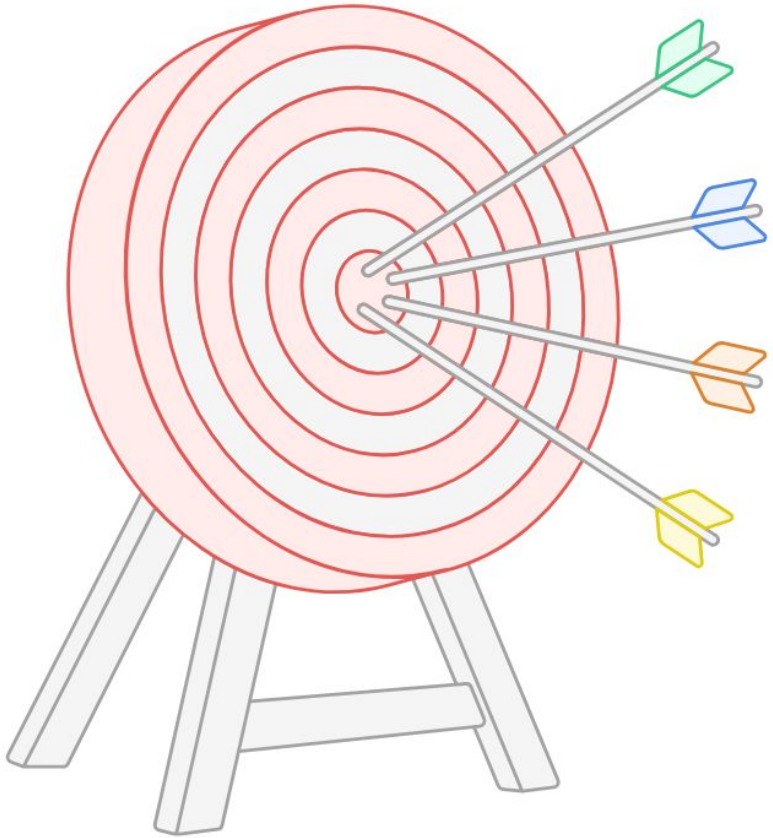
Representa la estructura del contenido de la página web como un árbol de nodos, donde cada etiqueta HTML es un objeto.

INTEGRACIÓN

Objeto Document

El objeto document sirve como punto central de interacción y es común tanto al BOM (DOM Level 0) como al DOM.

DOCUMENTACIÓN Y REFERENCIAS: W3Schools JS Reference | Especificación W3C DOM



DOM

Estándar W3C para la manipulación de documentos web

Objeto Document

Objeto central que representa el documento HTML

Nodos

Elementos individuales que componen la estructura del documento

Etiquetas HTML

Objetos que representan etiquetas HTML específicas

Actividad 0

¿Qué es el estándar W3C?



Actividad 0

¿Qué es el estándar W3C?

DEFINICIÓN

World Wide Web Consortium

Consorcio internacional que desarrolla recomendaciones y estándares para asegurar el crecimiento de la Web a largo plazo.

Fundado en 1994 por Tim Berners-Lee para promover la evolución ordenada de las tecnologías web.

PROPÓSITO

Interoperabilidad Web

Garantizar que todos los navegadores interpreten el código de forma homogénea e independiente de la plataforma.

Fomentar la accesibilidad universal, internacionalización y compatibilidad técnica.

APLICACIÓN

Estándares DOM / HTML

Define especificaciones universales para lenguajes web como HTML, CSS y el modelo de objetos DOM.

Proporciona un marco común para que lenguajes como JavaScript interactúen con la estructura web.

CUESTIÓN DE INVESTIGACIÓN: Investiga qué consecuencias tendría para los desarrolladores web un entorno sin estándares unificados como los del W3C.

Objeto Document

DOM Level 0 y el Browser Object Model (BOM)

ORIGEN

DOM Level 0

Es la primera versión del DOM, muy básica y antigua.

Surgió de las primeras implementaciones de JavaScript en los navegadores primitivos.

CONCEPTO

Modelo BOM

Internet Explorer y Netscape Navigator introdujeron el Browser Object Model (BOM).

Permite acceder y modificar las propiedades de las ventanas del propio navegador.

ESTRUCTURA

Objeto Window

El modelo BOM lo constituyen el objeto **window** y todas sus propiedades:

- **History**
- **Location**
- **Screen**
- **Navigator**

MAYOR INCONVENIENTE: Ninguna entidad se encarga de estandarizar el BOM o definir unos mínimos de interoperabilidad entre navegadores, lo que genera divergencias entre plataformas.

Objeto Document

Estandarización y Estructura DOM

DEFINICIÓN

El Estándar DOM (W3C)

Representa el **documento HTML** de la ventana actual. El **W3C** definió este estándar para **homogeneizar propiedades y métodos**, asegurando la **interoperabilidad entre todos los navegadores**.

Clave de acceso: SIEMPRE accederemos y modificaremos el objeto document mediante este estándar.

NIVELES DOM

DOM Level 1
Núcleo básico de navegación y manipulación del árbol HTML/XML.

DOM Level 2
Incorpora modelo de eventos, estilos CSS y espacios de nombres.

DOM Level 3
Soporte avanzado de carga, guardado y validación XML.

DOM Level 4
Estándar moderno, simplificación de eventos y Mutation Observers.

RECURSO Y SINTAXIS TÍPICA: document.getElementById("id")

Consulta la documentación completa y ejemplos interactivos en: **HTML DOM Documents**

- **MA03.01**: Clases con propiedades privadas usando #.
- **EX03.01**: Vídeos con la solución de cada ejercicio
- **¿Quien quiera saber como va la evaluación que lo diga?**
    - Vamos pasando uno a uno
- **¿Qué nos falta de la primera evaluación?**
    - AC04.01 Actividades
    - AC04.02 Actividades
    - **EX04.01**: ¿el jueves 12/11/2026 o antes?

Actividad 1

Formulario e Interacción DOM con JavaScript

ENUNCIADO

Objetivo Principal

Realiza una página html con un formulario y un script JavaScript para hacer lo siguiente:

Asegúrate de vincular los eventos adecuadamente y manipular los elementos del árbol DOM utilizando las funciones nativas.

REQUISITOS

1. Campo de texto
Un campo de texto con un valor inicial **«Texto inicial»**

2. Botón de lectura
Un botón tal que al pulsarlo muestre en un **alert** el valor del campo de texto.

3. Botón de edición
Otro botón tal que al pulsarlo cambie el valor del campo a uno solicitado al usuario mediante la función **prompt**.

DOCUMENTACIÓN Y CONSULTA DE REFERENCIA

Consulta en **w3schools** las propiedades de DOM input text acceder para obtener el valor seleccionado.

Actividad 2

Formularios y Selección de Valores en JavaScript

ENUNCIADO

Objetivo Principal

Realiza una página HTML con un formulario y un script JavaScript para interactuar con selecciones de usuario:

Asegúrate de acceder correctamente a las propiedades del objeto DOM para recuperar los datos elegidos.

REQUISITOS

1. Lista desplegable

Una lista desplegable (**select**) configurada con **3 valores de colores**.

2. Botón de confirmación

Un botón tal que al pulsarlo muestre en un **alert** el valor que está seleccionado en la lista desplegable.

DOCUMENTACIÓN Y CONSULTA DE REFERENCIA

Consulta en **w3schools** las propiedades de DOM **select** y **option** a las que debes acceder para obtener el valor seleccionado.

Actividad 3

Manipulación DOM con Select e Input en JavaScript

ENUNCIADO

Objetivo Principal

Realiza una página HTML con un formulario y un script JavaScript para realizar los ejercicios propuestos:

Asegúrate de acceder correctamente a los elementos del DOM y gestionar los valores dinámicamente.

REQUISITOS

1. Lista desplegable
Una lista desplegable (**select**) configurada con **4 opciones para ciudades**.

2. Campo de texto
Un campo de texto (**input**) inicialmente vacío.

3. Botón de transferencia
Al pulsarlo, toma el valor seleccionado de la lista y lo muestra en el campo de texto.

4. Identificación DOM
Utiliza el método **getElementById** para acceder a todos los elementos.

DOCUMENTACIÓN Y CONSULTA DE REFERENCIA

Consulta en **w3schools** las propiedades de DOM **select** y **option** a las que debes acceder para obtener el valor seleccionado.

Document.write()

Método del Objeto Document

DEFINICIÓN

Objeto Document

Es un método del objeto **document**, el cual representa la página web cargada en el navegador.

FUNCIONALIDAD

Escritura Directa

Sirve para escribir directamente código **HTML** o texto en la propia página web.

USO PRINCIPAL

Contenido Estático

Suele ser especialmente útil para insertar contenido estático de forma rápida.

Document.write()

Generación Dinámica de HTML

CONCEPTO Y CÓDIGO

Uso principal

El método **write** nos permite generar código HTML dinámicamente en el documento.

Ejemplo de sintaxis:

```
document.write('<p>El resultado es</p>');
```

CONSIDERACIONES

Condición de Uso

Sólo se debe usar si el usuario no va a interaccionar más con el nuevo HTML y no vamos a necesitar acceder a él posteriormente.

⚠ Inconveniente Principal

Origina el problema del **documento cerrado**, lo que dificulta notablemente su gestión y mantenimiento en aplicaciones dinámicas.

DOCUMENTACIÓN Y CONSULTA DE REFERENCIA

Más información y ejemplos prácticos disponibles en **w3schools.com (method document.write)**

Document.write()

Método del Objeto Document — Ventajas y Desventajas

VENTAJAS

Generación dinámica

Permite crear e inyectar código HTML o elementos dinámicamente en el documento en tiempo de ejecución.

Contenido estático

Es útil y directo para insertar bloques de contenido estático de forma rápida sin estructuras complejas.

DESVENTAJAS

Problemas de accesibilidad (A11Y)

Dificulta la lectura por parte de lectores de pantalla y herramientas de asistencia.

Problemas de rendimiento

Bloquea el procesamiento del DOM e impacta negativamente en la velocidad de carga.

Problemas de interacción

Si se ejecuta una vez cargada la página, provoca el problema del documento cerrado y sobrescribe todo el HTML.

AC04.01 - Actividad

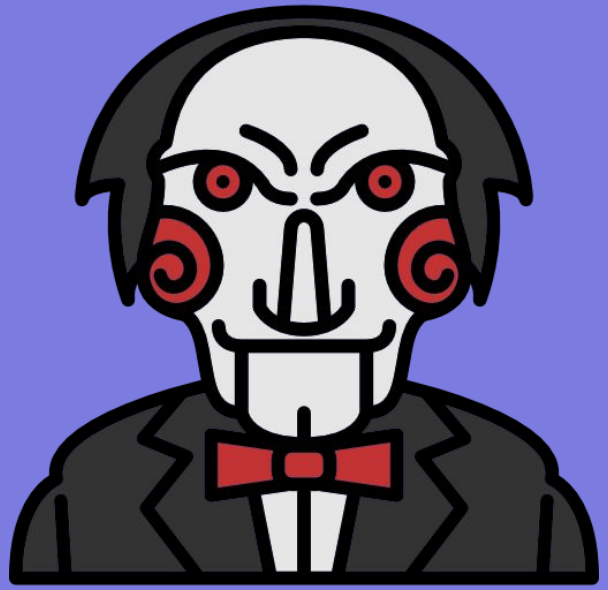


Trabajo Autónomo

Completa las actividades prácticas en la plataforma educativa:

- **Realizar todos los ejercicios del aula virtual `AC04.01`**

Continuará...



Objeto Window

Elemento Principal de la Jerarquía de JavaScript

Ventana del Navegador

Representa la ventana actual que contiene el navegador.

Objeto Raíz

Es el objeto raíz de la jerarquía de objetos de JavaScript.

Gestión Multiventana

Permite gestionar múltiples ventanas de forma independiente.

Objeto Implícito

No es necesario nombrarlo para acceder a sus propiedades:

window.document ó **document**

Recurso: w3schools - The Window Object

Objeto Window

Métodos y Propiedades del Objeto Global

Métodos		
alert()	forward()	setinterval()
back()	home()	setTimeOut()
blur()	moveTo()	scrollBy()
close()	open()	scrollTo()
confirm()	print()	stop()
find()	prompt()	setinterval()
focus()	resizeTo()	setTimeOut()

Propiedades		
closed	location	pageYoffset
defaultStatus	locationbar	parent
document	menubar	personalbar
frames	name	scrollbars
history	opener	self
innerHeight	outerHeight	status
innerWidth	outerWidth	toolbar
length	pageXoffset	top

Recurso: w3schools - The Window Object Reference

Gestión de las Ventanas

Características y Capacidades Principales en JavaScript

Gestión & Acciones

JavaScript permite gestionar diferentes aspectos relacionados con las ventanas, como abrirlas automáticamente o al presionar un botón.

Propiedades Únicas

Cada una de estas ventanas tiene un tamaño, posición y estilo visual totalmente independiente y diferente.

Contenido Dinámico

Las ventanas emergentes (pop-ups) suelen albergar información dinámica adaptada a la interacción del usuario.

Gestión de las Ventanas

Abrir y cerrar nuevas ventanas en la web

Uso Frecuente

Es una operación muy común y habitual en las páginas web.

Comportamiento Automático

En algunas ocasiones se abren de forma autónoma, sin requerir una acción explícita del usuario.

Control Limitado en HTML

HTML permite solicitar la apertura de nuevas ventanas, pero no proporciona ningún control posterior sobre ellas.

```
<a href="..." target="_blank">
```

Gestión de las Ventanas

Abrir y cerrar nuevas ventanas (Teoría)

1. Método open()

Con JavaScript es posible abrir una ventana vacía en el navegador utilizando este método:

```
let nuevaVentana = window.open();
```

2. Referencia a la Ventana

De este modo, la variable asignada (`nuevaVentana`) contendrá una referencia directa a la ventana creada para su posterior manipulación y control.

Gestión de las Ventanas

Apertura mediante interacción del usuario

Comportamiento en Navegadores Actuales

En la diapositiva anterior se exponía una teoría... que no se cumple en los navegadores actuales.
Sin embargo, si lo hacemos a través de un botón, la cosa cambia:

Ejemplo de Código

```
<button onclick="window.open('https://es.wikipedia.org/wiki/Faradio', '_blank', 'noopener,noreferrer')">
    Abrir página de Faradio
</button>
```

Gestión de las Ventanas

Abrir y cerrar nuevas ventanas (Sintaxis y Parámetros)

1. Parámetros del método open()

- **URL:** Dirección web del documento que se desea cargar en la nueva ventana.
- **Nombre de la ventana:** Identificador de la ventana o destino de navegación.
- **Colección de atributos:** Cadena de opciones que definen la apariencia y dimensiones (alto, ancho, etc.).

2. Ejemplo de Código

```
ventana = window.open("https://www.google.com", "Google", "height=100, width=100");
```

Crea una nueva ventana de 100x100px con el nombre "Google" cargando la URL indicada.

Documentación de referencia: w3schools - Window open()

Gestión de las Ventanas

Ejemplo de Apariencia y Código HTML / JavaScript

Código HTML del Ejemplo:

```
<body>
  <h1>Ejemplo de Apariencia de una Ventana</h1>
  <input
    type="button"
    value="Abre una Ventana"
    onclick="window.open('https://www.w3schools.com', 'w3schools', 'width=300,height=200')"
  />
</body>
```

Gestión de las ventanas

Apariencia y Posicionamiento

1. Dimensiones (Tamaño)

- Con las propiedades `height` y `width` podemos establecer el alto y ancho de las ventanas.
- Permite definir la escala inicial del contenedor.

2. Posicionamiento en Pantalla

- Métodos `moveTo()` y `moveBy()` para cambiar la posición.
- Se recomienda usar posiciones relativas con % de `screen.height` y `screen.width` según la resolución.

⚠️ Consideración Importante: Bloqueo de Pop-ups

Estos métodos pueden no funcionar adecuadamente debido a los bloqueadores de ventanas emergentes integrados en los navegadores modernos.

- Los navegadores suelen restringir el movimiento y cambio de tamaño no solicitado por el usuario.
- Es recomendable diseñar interfaces adaptativas que no dependan exclusivamente de múltiples ventanas secundarias.

Gestión de las ventanas

Comunicación entre ventanas

1. Control General

Desde una ventana se pueden abrir o cerrar nuevas ventanas.

2. Tipos de Ventanas

La primera se denomina ventana principal, mientras que las segundas se denominan ventanas secundarias.

3. Interacción

Desde la ventana principal se puede acceder a las ventanas secundarias.

📄 **Archivo de Ejemplo:** `ComunicacionEntreVentanas.html`

Actividad 4

1. Ejecución Práctica

Prueba `ComunicaVentanas` en tu ordenador.

2. Documentación

Documéntalo en un archivo **.pdf** aparte.

❓ Pregunta de Análisis

¿Qué problemas puede haber a la hora de ejecutarlo?
(Ej: bloqueador de ventanas emergentes, permisos del navegador, etc.)

Gestión de las ventanas

Comunicación entre ventanas: Secundaria → Principal

1. Objeto de Referencia

Desde una ventana secundaria podemos acceder a las propiedades y métodos de la ventana principal utilizando la propiedad `window.opener`.

2. Capacidad de Interacción

Permite manipular de forma directa el DOM, leer o modificar variables y actualizar elementos presentes en la ventana origen.

💻 Ejemplo de Código:

```
window.opener.document.getElementById('color').value = 'rojo';
```

Actividad 5

Requisito: Envío de datos desde la Ventana Principal a la Ventana Secundaria

01

Ventana Principal

- Crear un documento HTML base.
- Incluir una **lista desplegable** (`<select>`) con 3 opciones de colores.
- Añadir un botón etiquetado como **“selecciona color”**.

02

Evento y Apertura

- Capturar la selección realizada por el usuario.
- Al pulsar el botón **“selecciona color”**, ejecutar la apertura de la ventana.
- Abrir la ventana secundaria mediante un HTML previo.

03

Ventana Secundaria

- Disponer de un **campo de texto** (`<input>`).
- Recibir el valor de color desde la ventana de origen.
- Mostrar automáticamente el color seleccionado en el campo de texto.

Actividad 6

Requisito: Comunicación de la Ventana Secundaria a la Ventana Principal

01
Ventana Principal

- Crear un HTML con un botón llamado **“selecciona color”**.
- Incluir un campo de texto (`<input>`) inicialmente vacío.

02
Apertura Secundaria

- Al hacer clic en **“selecciona color”**, abrir la ventana secundaria.
- Cargar en ella un HTML previamente creado.

03
Ventana Secundaria

- Incluir lista desplegable (`<select>`) con 3 colores y opción por defecto **“- seleccione color -”**.
- Al seleccionar un color, este aparecerá en el campo de texto de la ventana principal.

El objeto Navigator

Permite identificar las características de la plataforma sobre la cual se ejecuta la aplicación web.

Información General

Ejemplos de datos que proporciona:

- Tipo de navegador
- Versión del navegador
- Sistema operativo

Enlace de interés:
useragentstring.com

Métodos

- **javaEnable()**

Propiedades

- **appCoreName**
- **appName**
- **appVersion**
- **cookieEnable**
- **platform**
- **userAgent**

Consulta de Referencia: Window Navigator

El objeto Screen

Especificaciones y dimensiones de la pantalla del usuario

Concepto General

- Corresponde a la pantalla utilizada por el usuario.
- Todas sus propiedades son **solamente de lectura**.

Casos de Uso

- Adaptar posición y tamaño de las ventanas emergentes según la resolución disponible.

Propiedades del Objeto

- availHeight
- availWidth
- colorDepth
- height
- pixelDepth
- width

Consulta de Referencia: Window Screen

El objeto History

Almacena las referencias de las páginas web visitadas.

Concepto General

- Las referencias se guardan en una lista para desplazarse entre las páginas web.
- No es posible acceder a las URL ya que es información privada.

Métodos

Métodos
back()
forward()
go()

Propiedades

Propiedades
current
length
next
previous

Consulta de Referencia: Window History

El objeto Location

Corresponde a la URL de la página web en uso.

Concepto General

- Corresponde a la URL de la página web en uso.
- Consulta las diferentes partes de una URL:
- El dominio
- El protocolo
- El puerto

Métodos

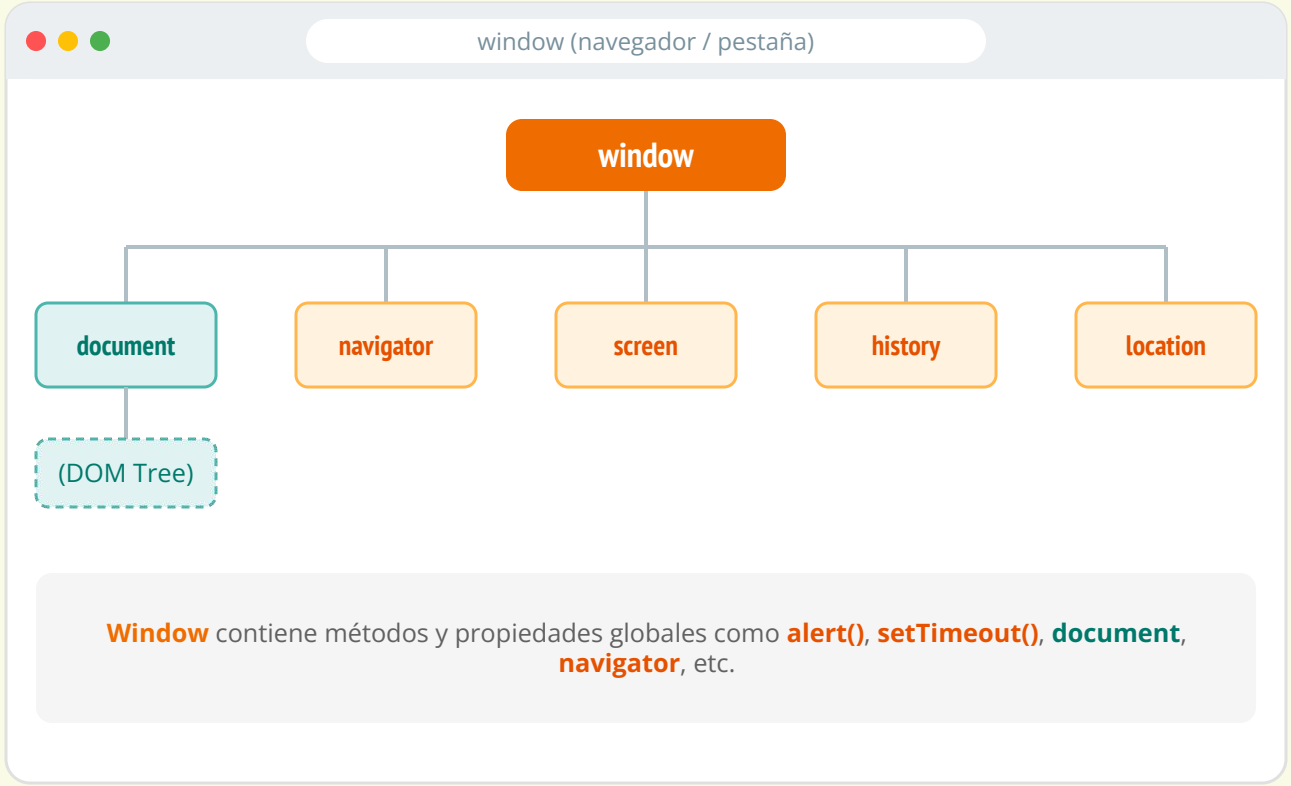
Métodos
assign()
reload()
replace()

Propiedades

Propiedades
hash
host
hostname
href
pathname
port
protocol
search

Consulta de Referencia: Window Location

Esquema


window (navegador / pestaña)
window
document
navigator
screen
history
location
(DOM Tree)
Window contiene métodos y propiedades globales como alert(), setTimeout(), document, navigator, etc.

¿Dudas? ¿Preguntas?

Si se os quedó algo en el tintero... ¡es el momento!

Turno de preguntas

Aprovecha para aclarar conceptos de la unidad, repasar detalles o resolver dudas sobre los ejercicios.

Debate abierto

Comparte tus reflexiones, sugerencias o enfoques alternativos con el resto de la clase.

AC04.02 - Actividad



Trabajo Autónomo

Completa las actividades prácticas en la plataforma educativa:

- **Realizar todos los ejercicios del aula virtual `AC04.02`**



FIN DE UT4

¡Enhorabuena! Hemos completado la Unidad de Trabajo 4

Objetos predefinidos